DATA 608 Project Proposal

Real-Time Energy Pipeline Early Warning and Monitoring System

Course

DATA 608 – Big Data Systems

Group: 1

Team Members

Rio Sibuea

Alejandro Alvarado

Ifeanyi Njoku

12-Mar-26

1. Problem Statement

Alberta's energy sector operates one of North America's most extensive pipeline networks, spanning thousands of kilometres through remote terrain and sensitive ecosystems.

Oil and gas pipelines transport large volumes of energy resources over long distances. Failures in these systems, such as leaks, pressure anomalies, or blockages, can lead to environmental damage, safety hazards, and economic losses. Early detection of such issues is critical for maintaining infrastructure reliability and preventing catastrophic failures.

Traditional monitoring systems rely primarily on threshold-based alerts derived from sensor readings such as pressure and flow rates. However, these approaches may not detect subtle anomalies or complex patterns that precede pipeline failures.

The goal of this project is to design and implement a real-time data pipeline capable of detecting abnormal pipeline conditions using streaming data and machine learning techniques. The system will simulate pipeline sensor data, ingest it through a streaming architecture, process it using distributed computing tools, and apply machine learning to detect potential anomalies. The output will be visualized through a monitoring dashboard that provides alerts and insights into pipeline conditions.

Key Value Proposition

Transform reactive pipeline monitoring into proactive, data-driven risk intelligence — reducing incident response time, lowering operational costs, and strengthening regulatory compliance with Canada Energy Regulator (CER) reporting obligations.

2. Proposed Method

The proposed solution is a real-time monitoring pipeline that processes streaming sensor data and applies anomaly detection techniques.

The system will consist of the following stages:

1. Sensor Data Simulation: A Python producer script will simulate pipeline sensor readings such as pressure, temperature, flow rate, and vibration under both normal and abnormal operating conditions.
2. Data Streaming and Ingestion: Simulated sensor events will be streamed into the system through a data streaming service such as AWS Kinesis.
3. Data Processing: The incoming data stream will be processed using AWS Lambda to perform feature engineering and data aggregation.
4. Machine Learning Anomaly Detection: A machine learning model will analyze the processed sensor data and generate anomaly flags or risk scores.
5. Data Storage: Raw and processed data will be stored in a data lake on Amazon S3 using the Parquet format.
6. Visualization and Alerts: A monitoring dashboard built with Streamlit will display real-time sensor readings and system health indicators. Amazon SNS will process

the anomaly alerts and send them to the corresponding personnel which will attend to the pipes (e.g. email or sms).

3. Data Sources and Data Generation

The primary datasets that will be used for this project will be oil pipeline sensor data collected from the Canada Energy Regulator (CER) and from Kaggle. This data will be used for the training, testing and validation of the classification machine learning model that will identify the anomalies.

Below are the dimension of the datasets:

- The dataset from Kaggle has 14 columns and 1000 rows.
- The CER dataset has 1997 rows and 102 columns.

The datasets are publicly available for access.

Additionally, the team will employ data simulation techniques in order to artificially create a pipeline sensor data stream using a Python producer to put the monitoring system to the test.

The simulator will generate sensor readings at fixed time intervals (one record per second). Each record will contain information such as timestamp, pipeline identifier, segment identifier, pressure, temperature, flow rate, vibration, and a simulated condition label (anomaly or normal).

The simulator will generate multiple operating scenarios including:

- Normal operation
- Potential leakage/blockage/pressure spike scenarios (Anomaly)
- Critical state

The simulated anomalies will be injected by modifying normal sensor readings according to predefined patterns. For example, leaks may cause pressure and flow to decrease, while blockages may increase pressure and decrease flow rates.

Because the data is simulated, care will be taken to ensure realistic value ranges and noise levels to mimic real-world sensor variability. The inclusion of ground-truth labels will allow evaluation of the anomaly detection model.

4. Data Storage Strategy

The project will use data lake architecture to store both raw and processed data.

Storage Platform:
Amazon S3 will be used as the primary storage system.

File Format:
Data will be stored in Parquet format. Parquet provides columnar storage, efficient compression, and high performance for analytical workloads.

Storage Layers:
- Raw Layer: stores the original sensor events received from the stream.
- Processed Layer: stores cleaned data and engineered features.
- Analytics Layer: stores aggregated datasets used by the dashboard.

This layered architecture improves scalability, reproducibility, and performance for large-scale analytics workloads.

5. Tools, Skills, and Infrastructure

Programming Language:

- Python

Big Data Tools:

- AWS Lambda
- AWS Kinesis

Cloud Infrastructure:
The project will use Amazon Web Services (AWS) for cloud infrastructure, including:

- AWS Kinesis for streaming ingestion
- Amazon S3 for data storage
- Amazon Lambda for data processing
- Amazon SNS for alert/notification system

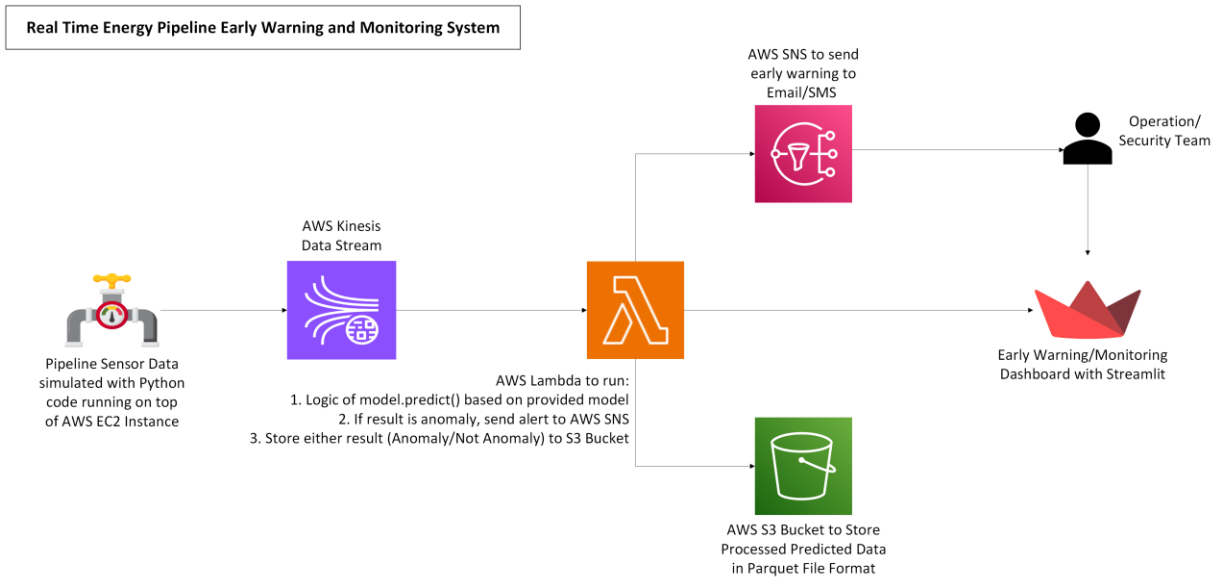
Machine Learning Tools:
Scikit-learn will be used for building anomaly detection models such as Isolation Forest or Logistic Regression.

Visualization Tools:
Streamlit will be used to build a monitoring dashboard for displaying real-time pipeline metrics and anomaly alerts.

6. System Architecture Overview

The system will implement a streaming data pipeline where simulated pipeline sensor events are continuously generated and ingested into the system. These events will be processed by a distributed computing framework that performs feature engineering and machine learning inference. The processed data and model predictions will be stored in a cloud-based data lake and displayed through a monitoring dashboard.

This architecture demonstrates the integration of streaming ingestion, distributed processing, cloud storage, and machine learning for infrastructure monitoring.



Created by Rio Sibuea with Visio

6.1 Architecture Components

Layer	Technology	Role
Sensor / EC2	**Python Simulator on EC2**	Generates pipeline sensor readings every specific interval and streams to Kinesis
Ingestion	**Amazon Kinesis Data Stream**	Aggregates from multiple sensor data. Buffers real-time telemetry; 1 shard, us-east-1 region
Processing	**AWS Lambda (Python 3.11)**	Runs model prediction, classifies anomalies, triggers SNS, writes to S3
Alerting	**Amazon SNS**	Sends early warning emails/SMS to Operation/Security Team on anomaly
Storage	**S3 Parquet (awswrangler)**	Stores all predicted results in Parquet format, partitioned by date/Pipe ID
Dashboard	**Streamlit + Plotly**	Early Warning / Monitoring Dashboard — auto-refreshing, dark-themed
ML Model	**scikit-learn**	Trained on real CER + KAGGLE data; classifies Normal/Anomaly/Critical

7. Success Criteria and Evaluation

The success of the project will be evaluated using both system performance metrics, model performance metrics and deliverables.

System Performance:

- Ability to process streaming data in near real time
- Stability and reliability of the data pipeline
- Latency between data ingestion and dashboard updates

Machine Learning Performance:

- Precision and recall for anomaly detection
- Detection rate of simulated anomalies
- False positive rate

Deliverables:

- Data can flow through the pipelines, stored into storage and presented in the Dashboard
- The machine learning model can notify an anomaly when there is potential risk
- Anomaly notification is delivered to recipients (email)

A successful project will demonstrate a working real-time data pipeline, automated anomaly detection, and an operational monitoring dashboard.

8. Team Responsibilities

The project responsibilities are divided among team members according to the team contract.

Rio Sibuea:

- Develop and manage the ingestion of sensor data into the system
- Implement and manage the streaming architecture
- Ensure reliable data flow.

Alejandro Alvarado:

- Develop the machine learning model for pipeline anomaly detection
- Optimize model performance and evaluate accuracy.
- Integrate the trained model into the real-time pipeline.

Ifeanyi Njoku:

- Develop the monitoring dashboard using Streamlit
- Visualize sensor data and anomaly detection results.
- Implement alert notifications.

Shared Responsibilities:
All members will participate in system design decisions, review each other's work, assist with implementation challenges, and contribute to documentation and the final presentation.

9. Expected Outcome

The final system will demonstrate how real-time big data pipelines can be used to monitor energy infrastructure.

The project will produce:

- A functioning streaming data pipeline
- A machine learning-based anomaly detection system
- A monitoring dashboard displaying pipeline health indicators
- Insights into pipeline behavior under normal and abnormal operating conditions

This project highlights the potential of combining big data technologies and machine learning to improve infrastructure safety and operational monitoring.

10. Implementation Plan

The implementation of this solution will consist of 5 phases:

Phase	Title	Duration	Key Deliverables
Phase 1	Data & Environment Setup	1 week	CER + KAGGLE datasets ingested; EDA notebooks complete
Phase 2	Model Development	3 days	Model trained and validated
Phase 3	Data Pipeline Infrastructure	2 weeks	Kinesis + Lambda + S3 Parquet + SNS alerting end-to-end
Phase 4	Dashboard & Integration	1 week	All Streamlit panels live
Phase 5	AWS Deployment	1 week	AWS deployed; IAM roles; load testing; documentation finalised

Note: Project phases are running concurrently rather than sequentially

11. Cost Estimate (AWS Production)

The architecture is designed for minimal AWS spend

Service	Est. Monthly Cost	Notes

Amazon Kinesis (1 shard)	**~$14 CAD**	us-east-1
AWS Lambda	**~$2 CAD**	Micro-batch; within free tier for dev/test
Amazon S3 (Parquet storage)	**~$3 CAD**	
Amazon SNS	**~$1 CAD**	Email/SMS alerts
Data transfer / misc	**~$5 CAD**	Dashboard reads, CloudWatch logs
TOTAL	**~$25/mo**	**Scales linearly with sensor count**

References

Incidents at CER-regulated pipelines and facilities. Open Government Portal. (n.d.). https://open.canada.ca/data/en/dataset/fd17f08f-f14d-433f-91df-c90a34e1e9a6

Waqas, M. (2025, May 29). *Pipeline dataset in oil and Gas Sector/Fertilizer*. Kaggle. https://www.kaggle.com/datasets/muhammadwaqas023/pipeline-dataset-in-oil-and-gas-sector

AWS product and Service Pricing - Amazon.com. (n.d.). https://aws.amazon.com/pricing/

Amazon. (n.d.-a). https://docs.aws.amazon.com/